Identifying Homogeneous IoT Devices with Hybrid Locality Sensitive Hashing

Jay Thom, Nathan Thom, Batyr Charyyev, Emily Hand, Shamik Sengupta
Department of Computer Science and Engineering, University of Nevada, Reno, USA
1664 N. Virginia street m/s 0171 Reno, NV 89557 775-784-6905
Email: [jthom, bcharyyev, emhand, ssengupta]@unr.edu, nathanthom@nevada.unr.edu

***Abstract*—Internet of Things (IoT) devices have become increasingly prevalent, and while convenient they pose security risks and must be monitored to keep networks safe. The problem of identifying IoT devices by fingerprinting their network traffic has been studied, with various approaches emerging. While achieving good results, many solutions require complex feature extraction, considerable computational overhead, and extensive domain knowledge in both networking and machine learning to select relevant features and supporting ML algorithms. In addition, many current studies work to identify heterogeneous devices in an artificially sterile (lab) environment. To improve the process we introduce FlexHash, a system that uses a combination of locality-sensitive hashing and machine learning to identify specific devices based on a generic view of their network traffic characteristics. We successfully identify both heterogeneous and homogeneous (identical) devices in an environment with both live network noise and noise generated from other (unknown) IoT devices. FlexHash is able to consume unprocessed network traffic in the form of .pcap files and produce feature vectors capable of highly accurate device identification and anomaly detection. To enhance the strength of this approach we develop our own n-gram based hashing method allowing for various parameters in the algorithm to be tuned, making it possible to accurately differentiate individual devices from among a field of identical peers as well as device genre and heterogeneous devices with a single packet of network traffic.**

***Index Terms*—Internet of Things, IoT Device Identification, Network Traffic Fingerprinting, Similarity Hashing**

I. Introduction

In recent times the concept of an Internet of Things (IoT) has been growing with numerous devices such as cameras, low-powered sensors, wearable technologies, and a multitude of household, commercial, industrial, and medical devices and sensors inter-connected to one another and to the Internet. The current digital landscape is essentially surrounded by IoT devices which play increasingly significant roles in our lives and have access to our private networks and data. This is a trend which is likely to continue, with reports showing there are as many as 30 billion IoT devices connected worldwide [1] with continued rapid growth expected. Along with the convenience offered by these technologies, there are also increased security risks with many devices under-equipped to prevent compromise, leaving themselves and the networks to which they are connected vulnerable to attack.

A first step in providing security for these systems and the networks on which they reside is proper accounting and management; tracking known devices, discovering rogue devices, and identifying abnormal or anomalous device behavior. This, however, can prove to be a challenging task. Common device identifiers such as IP and MAC addresses are of little use as they can be easily spoofed, and devices can connect to networks surreptitiously through rogue WiFi hot spots, requiring the development of alternate methods. The topic of IoT device identification by analyzing generated network traffic has been a popular area of research recently with several proposed solutions found in the literature. Various approaches including statistical, rule-based, and machine learning techniques have been explored which collect and analyze network traffic either in real-time or offline. Rule sets or models for machine learning classifiers are developed which have been able to identify specific devices and device types as well as differentiating *normal* traffic from *anomalous* traffic. This has also been helpful for highlighting when new or unknown devices connect, or when familiar devices begin behaving in unexpected ways; an indication of compromise. Many of these methods are not without their weaknesses, however, requiring specific traffic intervals be collected, that adequate quantities are collected, or that classifiers are able to operate within available computational constraints. Most require feature extraction and engineering which can introduce a high degree of overhead, or high degrees of required domain knowledge to choose the appropriate features for the given classifiers. In addition, many classifiers require models be updated and retrained frequently to continue providing good results [2] [3].

While good results have been achieved, previous methods for IoT network traffic fingerprinting often focus on *heterogeneous devices* in a laboratory environment devoid of the *background noises* present in live networks. Many studies have relied on data sets provided by seminal studies such as IotSentinel [4], which are made up either of completely different (heterogeneous) devices or of different types of devices from the same manufacturer. In a realistic network setting, an administrator will likely need to ***track many devices of different types, but also multiple devices of identical make and model (i.e. many identical web cameras) all producing very similar traffic. In addition, this traffic will be mixed in the presence of background traffic noise, possibly from other unknown or un-modeled IoT devices.*** In our previous work [5] we found that while our initial efforts were highly

effective with heterogeneous devices they performed poorly against groups of identical (homogeneous) devices on the same network, leading us to this current work.

Locality-sensitive hashing is useful in performing similarity searches and has proven to be of fundamental importance in numerous fields, such as spam email, machine learning, data mining, and malware detection [6]. In a similarity search, "given a collection of objects D with some similarity measure s defined between them and a query object q, retrieve all objects from D that are most similar to q according to the similarity measure s" [7]. Put more simply, a hash can be generated that is very similar for similar inputs, with minor changes causing only slight differences in the resultant digest. This is in contrast to the more familiar cryptographic hash, wherein the changing of a single bit in an input will cause a complete change in the output. For IoT identification, a model can be built from several samples of traffic generated by the same device, stored in a database, then compared to previously unseen traffic to find the highest degree of similarity to approximate a match. In the same sense, a threshold can be selected to identify devices of similar type, or genre. This approach provides a means for device identification without the need for the complexities in the feature selection and extraction of machine learning-based approaches. Feature selection and extraction from data is an important step in machine learning-based traffic fingerprinting as it directly impacts the accuracy and overhead of the system.

In this paper, we present FlexHash, a novel system combining the data generalizing capability of locality-sensitive hashing with the power of machine learning to achieve a highly accurate network traffic fingerprinting model for device identification. FlexHash uses only a single packet from network traffic and also achieves high accuracy in noisy environments. It also achieves high accuracy in identifying devices from groups of identical devices, a challenging task for traditional fingerprinting methods as identical devices are expected to generate similar network traffic. FlexHash avoids the complexity of feature selection and engineering by hashing an entire packet and then converting the resultant digest directly into a feature vector. Subtle differences in the typically repetitive traffic produced by IoT devices are captured by similarity hashing, and can then be modeled with the help of machine learning algorithms to produce a device fingerprint. The ability to achieve high accuracy using only single packets for device identification will allow for system monitoring by periodically sampling traffic to detect device membership, device genre, the presence of unknown devices, and changes in device behavior indicating traffic anomalies and possible compromise.

Contributions of this work are as follows:

- We provide an extensive review of related IoT network traffic fingerprinting approaches found in the literature.
- We develop FlexHash, a novel locality-sensitive hashing algorithm that enables adjustments to the hashing parameters (accumulator length, window size, and n-grams).
- We implement a network traffic fingerprinting method combining FlexHash with machine learning, and perform accurate IoT device identification requiring only a *single packet* of network traffic.
- We evaluate this system by classifying device genre and *identical devices* in the presence of identical peers while also including realistic *background noise*.
- We collect traffic data from three categories of 8 identical IoT devices, which we share with the research community at *github.com/UNR-IoT-Fingerprinting/FlexHash*.

The remainder of this paper is organized as follows: section-II provides a review of related works, section-III will describe our methodology and details of FlexHash, section-IV will provide experimental results and an analysis of our method, and finally section-V will present a conclusion and future work.

II. Related Works

In this section, we will provide existing works on network traffic fingerprinting of IoT devices. The summary of the studies is provided in Table I. Overall we categorized studies into two broad categories such as machine learning-based approaches and non-machine learning-based approaches and each is divided into subcategories based on the focus and algorithm used in the studies.

A. Non-ML Approaches

Studies in this category focus on temporal features [8], [9] and rule-based feature extraction [10], [11], and hashing the traffic data [12]. Authors focus on communication features found in packet header information [8]. Devices are identified by comparing traffic from a target device with previous samples stored per device in a database and using the similarity between samples. The similarity is measured by digitizing features and then comparing their Euclidean distance. [9] seek to characterize IoT traffic in terms of temporal patterns, volume, and target endpoints, and also consider security and privacy concerns. [11] seek to build annotations for device types, vendors, and product names by leveraging application layer response data from IoT devices and relevant websites to obtain product descriptions. [10] develop a baseline normal profile using traffic discovery and classification, then work to detect traffic anomalies based on the following features; overall traffic, transport layer protocol traffic, traffic by destination socket, packet size, and IP flow. [12] use locally sensitive hashes to determine similarity in network traffic flows, a useful technique in that feature extraction is not required. Hashes are generated and compared for similarity using the Nilsimsa hashing tool, achieving a high degree of precision and recall when compared against a set of devices used to generate traffic flow samples in the same network.

B. ML-Based Approaches

In machine learning-based approaches previous studies mainly extract traffic features from different components of the data such as header [13], [14], payload [15], or different states of the device such as initial connection setup phase [4], or idle/active state of the device [16]. Then studies use these

TABLE I: Summary of related works based on identification method and key feature set

Approach	Focus	Algorithm	Ref	Performance	Details
Non-Machine Learning	Temporal Feature Extraction	Euclidean Distance	[9]	75% accuracy with packet header information. 89% with TCP header and a lower layer header or using HTTP header and lower layer in 4 homogeneous devices.	Focus on communication features found in packet headers. Similarity between samples measured by digitizing features then comparing Euclidean distance. The most unique features found in device traffic are given a greater weight and are used to identify traffic from the same device. Limitations can occur based on OS or software updates or changes in the network.
		Comparison with expert rule set	[10]	Study focuses on dataset development using traffic from an unspecified number of home IoT devices.	Devices characterized by temporal traffic patterns, traffic volume, and target endpoints. Focus is on developing a dataset of live devices rather than devices in an artificial lab environment. Fingerprinting driven by an expert rule set.
	Rule-Based Feature Extraction	Statistical Analysis	[11]	No accuracy measurement. Study used a large body of botnet traffic.	Seeking to identify anomalous traffic, a baseline for normal behavior is built by traffic discovery and classification. Features such as overall traffic, transport layer protocols, dest. socket, packet size, and IP flow identify anomalies.
		Natural Language Processing	[12]	Rule precision between 95.7% and 97.5% with a large body of heterogeneous and homogeneous devices.	Authors build annotations for device type, vendors, and product names by using application layer response data from IoT devices and relevant websites to obtain product descriptions, natural language processing and data mining are incorporated to drive an Annotation Rule-Based Engine (ARE). Annotations are used to build a rule set to control and restrict communications.
	Device Traffic Hashes	Bitwise Difference	[6]	Avg. precision and recall of 93% and 90% respectively using 22 heterogeneous devies.	Hashes of device traffic flows are generated using Locality Sensitive Hashing. Similarity metrics are used to determine whether traffic samples were produced by same device. Flow lengths are tested to find minimally accurate sample size.
Machine Learning	Temporal Feature Extraction	CNN	[13]	86.7% accuracy achieved in 2 heterogeneous devices.	Heterogeneous traffic is collected using a Raspberry Pi. Device fingerprinting based on packet inter-arrival time with graphs generated by a CNN.
	Network Layer Features	Supervised Learning	[14]	Overall accuracy of 99.281% across 13 heterogeneous devices.	Two stage classifier; first stage identifies 'IoT' or 'Non-Iot'. Second stage attempts to identify one of 13 heterogeneous devices. Features are extracted from packet flows based on source and destination IP from SYN to FIN. Data is enriched with Alexa Rank and GeoIP information.
	Application Layer Features	Decision Trees	[1]	Authors claim 100% precision, recall and F-score for 3 heterogeneous devices.	Application layer features such as ARP, SSL, LLC, EAPOL, HTTP, MDNS, and DNS are captured to develop a static view of device behavior.
	Initial Connection Phase	Random Forest	[4]	Average accuracy of 81.5% in a set of 27 heterogeneous devices.	A fixed set of link and application layer features are captured during device initial connection phase to construct a device gateway to limit communication.
	Device Genre Identification	Unsupervised Clustering	[15]	98.2% accuracy with 33 heterogeneous devices.	Authors define policies on device type rather than on specific devices. Traffic is collected after initial connection phase to identify device classes.
	Flow Based Feature Extraction	Ensemble Approach (multiple)	[16]	94.5% accuracy for device type, 93.5% accuracy for traffic classification, 97% accuracy for the identification of anomalous traffic utilizing 7 heterogeneous devices.	Statistical analysis applied to flows of 16 consecutive packets, extracting 4 simple features; packet size, traffic direction, timestamp, and transport protocol. Utilizes decision trees, recurrent neural networks, random forest, residual neural networks, and convolutional neural networks.
		K-Means	[17]	Overall accuracy of 94% in a set of 10 heterogeneous devices.	Set of one-class clustering models from a real time flow level telemetry. Primary features are average packet size and average flow rate.
		Clustering and Deep Neural Networks	[18]	Avg. accuracy of 91.2%, 92.9%, and 81.8% with input features of 297, 234, and 179 respectively in 10 heterogeneous devices.	Clustering is combined with deep learning to classify devices. 297 features are extracted and used to build a classifier. An auto-encoder is used to reduce the dimensionality of the data set. Results are used to build a device white list.
		Ensemble Approach (multiple)	[19]	White listed devices identified with an accuracy of 90.3% using 27 heterogeneous devices.	67 features are extracted from a series of packets in a network traffic flow to build a one class classifier. Algorithms include adaptive boosting, latent dirichlet allocation, KNN, decision tree, naive bayesian, SVM, random forest.
	Image Analysis	Representation Learning	[20]	99.86% accuracy using 10 heterogeneous devices.	Device traffic is collected from single TCP sessions. The hexadecimal value of the packet payload is taken from a .pcap file and is converted into an image. Image serves as a fingerprint for identifying traffic from the group of 10 devices.
	Single Packet Identification	Genetic Algorithm	[21]	95% accuracy in a dataset of 23 heterogeneous devices.	212 features are extracted from 23 devices and identification is done using single packets. Genetic algorithm is used to identify relevant features.
		WEKA ML Analysis Tool (various)	[22]	46 heterogeneous devices, achieves an average accuracy of 97.78%.	Features are extracted from TCP headers. Feature vectors are developed by generating a score for all available features based on variability, stability, and suitability of each bit. WEKA tool is then utilized to sample ML algorithms.
	Behavioral Analysis	Scikit Learn (various)	[23]	14 heterogeneous devices are tested yielding an average accuracy of 99%. 5-packet min.	To answer the questions "what is this device" and "is this device what it claims to be", the authors perform device fingerprinting by extracting network protocols used from packet headers as well as information from payloads. They also observe command/response sequences from device to smart phone.
		Ensemble Approach (multiple)	[24]	Accuracy between 74.12% and 93.91% in a set of 6 sensors.	Packet header information is analyzed for statistical information which is combined with sensor data to create a feature set. Method is tested in normal and under attack scenarios. Seven behavioral profiles are built to develop device access rules. ML algorithms are chosen based on linearity of the captured data.
	Limited Feature Extraction	Semi-Supervised Learning	[25]	An average accuracy of 99.81% is accomplished using a set of 25 heterogeneous devices.	Rather than determining a large set of features, a smaller set is used along with a larger body of unlabeled data, with labels inferred from the smaller set. Features extracted include time intervals, traffic volume, protocols, and transport layer security features. CNN's are used for feature space reduction.
	Textual Features and Data Mining	Bag of Words	[26]	31 of 33 heterogeneous devices correctly identified.	Features are extracted from packet service discovery protocols, DHCP fingerprinting, and user agent information from HTTP headers. Vectors are added to a database forming a matrix where columns represent words and rows represent devices.
		Decision Tree	[27]	97% average accuracy in a 10 heterogeneous devices.	Features are extracted from a network flow as well as textual features from packet payloads. Packets are collected as devices join a network and performs its connection setup phase. Features are modeled as a bag-of-words with each device description making up a feature vector.
	Feature Cost Metrics	Various Algorithms	[28]	Average accuracy of 96.8% using 15 heterogeneous devices.	Authors focus on the cost of selecting appropriate features for developing a classifier. Features are selected on the basis of their utility for meeting the needs of specific algorithms. 111 features are collected from a set of 15 devices.

extracted features with different machine learning algorithms to fingerprint the traffic. The main traffic header features are the inter-arrival timing of packets [13], port numbers [14], existing protocols in traffic data [1], packet size [17], [18], flow rate [18]. Other traffic features used by the studies are textual features extracted from DHCP [19], XML description and mDNS information [20]. The main machine-learning algorithms are Convolutional Neural Networks (CNN) [13], random forest/decision tree classifiers [4], [14], one class classifiers [18], K-Means [18] and ensembling multiple methods [17], [21]. Other methods used by the studies are Bag of Words with textual data [19], [20]. Authors also focus on multi-stage classification such as in the first stage of the classifier categorizes traffic as *IoT* or *non-IoT* traffic and the subsequent second stage further attempts to categorize devices by device type [14]. Previous studies also focus on the amount of data used for fingerprinting. [4] focuses on extracting features from the first 13 packets in the connection phase and [22] focuses on extracting traffic features from a single packet using genetic algorithms. Since the feature vector used in the fingerprinting can impact the accuracy and overhead of the system, previous studies also focus on extracting relevant features from data [22], [23]. Previous studies use feature selection based on the feature utility [24], and optimization methods such as genetic algorithms [22].

C. Difference of Our Study from Previous Works

Non-ML approaches offer the advantage of simplicity, but often require larger traffic samples for comparison, and potentially large databases which must be searched to find matching device profiles. Machine learning solutions offers the advantage of highly accurate identification with smaller traffic flows and lend themselves to real-time monitoring which is more difficult when a database is required for comparisons, when larger flows are required, or when rules must be applied for identification. While effective, machine learning approaches rely on feature extraction, selection, and engineering, a process that can be computationally expensive and typically requires a high degree of domain knowledge both in machine learning and networking to select appropriate and relevant features for a given algorithm. FlexHash takes advantage of both approaches by combining a hybrid similarity hashing technique with machine learning, giving it the strength and real-time monitoring capability using only a single packet of network traffic for highly accurate identification, but requires no feature extraction or engineering.

Overall our work is different from existing studies in the following aspects:

- Our method does not depend on feature selection from the data which is computationally expensive and requires domain knowledge.
- We mainly focus on fingerprinting homogenous (identical) devices rather than heterogeneous (devices with different manufacturers and types) devices. Note that the identification of homogenous devices is a complex problem compared to the identification of heterogeneous devices as similar devices are expected to have similar traffic characteristics.
- Our approach works when there exists traffic noise, which is an important aspect as in real-world deployments the traffic noise is unavoidable.
- Finally our method can be tuned to fit various networks with different network characteristics.

Our work addresses some of the main challenges in traffic fingerprinting such as identification of homogenous devices, identification in noisy environments, and feature selection from the network traffic data.

III. Methodology

FlexHash relies on the principle of Locality Sensitive Hashing (LSH) [25] to convert *pcap* files into feature vectors. The resultant vectors are suitable for use with machine learning algorithms to produce IoT network traffic fingerprints. Since we use the hash of the data as input (feature vector) for machine learning algorithms we do not have to select and extract relevant features from the header or payload of the traffic data. In this section, we will provide a brief background on LSH and provide details of the proposed hashing method *FlexHash* and show how it is used in combination with machine learning algorithms to identify IoT devices. We will also provide details of the data set used for evaluation.

A. Locality Sensitive Hashing

Unlike cryptographic hashes which produce an entirely different output if even a small change is made in the input data, locality sensitive hashing produces similar outputs for similar inputs. If a small change is made in the input data, only a small change will result in the output. LSH has been utilized for a number of tasks including electronic identification [26], anomaly detection [27], malware classification [28], and spam email detection [29] [30]. Nilsimsa [29], ssdeep [31], sdhash [32], and tlsh [33] are the most commonly known n-gram based LSH approaches. In general, these hash functions operate by using a sliding window of a specific size that moves over the input stream one byte at a time. In each iteration, trigrams are generated from the current characters in a window which are then passed to a hash function to generate integer index values. Using these values, frequencies of the observed trigrams in the input are stored in an accumulator array and a hash digest is created based on these stored values.

We must have the ability to adjust the parameters of the LSH algorithm such that its functionality can be tuned to allow optimization of the output for individual devices. In our previous work [5], we find that Nilsimsa works well for fingerprinting heterogeneous devices exhibiting clearly different network behavior, but is less effective when used to hash traffic for identical devices. For this reason, we develop a new n-gram based hashing method, namely FlexHash which can be tuned and used to fingerprint network traffic of the IoT devices.

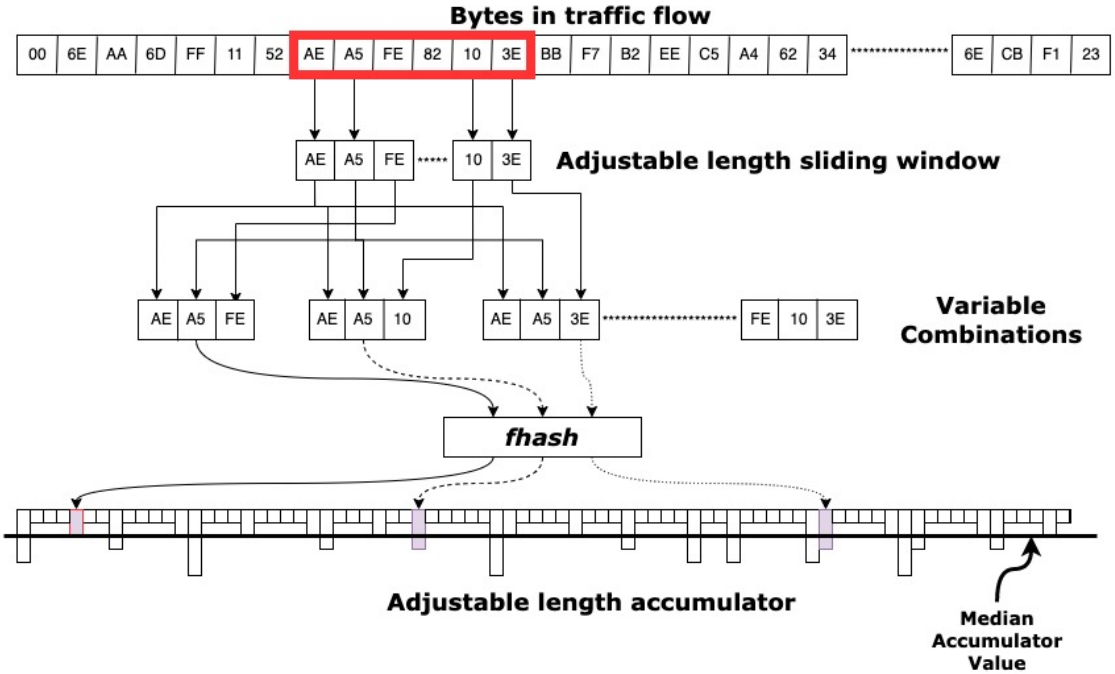


Fig. 1: FlexHash functionality with tunable parameters

B. FlexHash: Hybrid LSH

Compared with other n-gram based approaches (i.e. *Nilsimsa*, *TLSH*), FlexHash provides greater flexibility by allowing parameter tuning (sliding window, n-gram, and digest length) to be used with specific devices.

Table II provides the tunable parameters along with the set of ranges chosen for each parameter for this study. Details of FlexHash's functionality are provided in Figure 1. FlexHash utilizes an adjustable size sliding window that reads the input one byte at a time. At each iteration, all combinations of bytes in the window are generated and are passed to the hash function *fhash*. This function (fhash) multiplies the cumulative integer value of each combination. It then adds a prime number to the result to facilitate greater hash distribution. The result is divided by the length of the accumulator and the remainder (modulo) becomes the current index. The value in that index of the accumulator, which is initialized at zero, is incremented by one. As a result, the accumulator holds frequencies of the hashes of each observed combination taken from an input. The final digest of the input will be the same length as the accumulator. Finally, for every position (address) in the accumulator, if the value at a given address exceeds the median value of all indexes then the value of that address is set to 1, otherwise it is set to 0 to create a digest. The window size and accumulator size are adjustable and can be tuned based on the problem and input characteristics. The n-gram (combination) size is also adjustable with the upper limit being the current window size. Pseudo code for this process is presented in Algorithm-1. To perform optimal parameter selection for a particular device, we run pre-chosen combinations of parameters and test

TABLE II: Tunable Parameters of FlexHash

Accumulator Size (in bits)	128, 256, 512, 1024
Window Size	4, 5, 6
Combination Size	[2: Window Size]

which set provides the best results. Optimal parameters vary based on the device type, making the adjustable property of FlexHash vital to improving accuracy and other performance metrics, especially when identifying identical devices. Optimal parameters for devices found in our evaluations are *1024, 6, and 2* for accumulator, window, and combination size for smart plug and light bulb, and *1024, 4, and 2* for web cameras.

One important aspect of FlexHash is data must be hashed such that there is an even distribution of hash collisions across the length of the accumulator. In FlexHash this is achieved by *fhash*, which receives a set of combinations and returns a hash value in the range (0, [accumulator length]-1) which represents an array address. Segments of the data that are sufficiently similar will output the same (or similar) addresses. This allows FlexHash to capture the essence of device behavior, especially as it relates to repetitive activities which are common in IoT devices. Pseudo code for the process (fhash) of hashing the data and producing appropriate list indexes is presented in Algorithm 2.

C. Hashing traffic data

As mentioned earlier, FlexHash avoids the complexity of feature selection and instead applies the entirety of a traffic data in hashed form to produce a fingerprint. Earlier studies such as [1] [9] [11] [13] often work to capture network features from packet headers such as protocol, packet size, packet inter-

Algorithm 1 FlexHash

```
x ← window size
y ← ngram size
z ← accumulator size
procedure MAKEHASH(x, y, z)
    for .pcap in file do
        counter ← 0
        for bytes in .pcap do
            current ← window[counter:counter+x]
            ngramlist ← current
            procedure FHASH(ngramlist)
                return index
                accumulator[index += 1
            end procedure
        end for
    end for
    total ← 0
    while index < z do
        total ← total + accumulator[index]
        median ← total / z
        for accumulator[x]=0; x < z; x++ do
            if accumulator[index] ¡ median then
                accumulator[index] ← 0
            else if accumulator[index] ¿ median then
                accumulator[index] ← 1
            end if
        end for
    end while
    vector = []
    counter ← 0
    for bits in accumulator do
        current ← accumulator[counter:counter+8]
        current ← int(current)
        vector.append ← current
        counter ← counter + 8
    end for
end procedure
```

Algorithm 2 fhash

```
ngram list ← current ngrams
total ← 1
for each ngram do
    total ← total * ngram
end for
result ← total + 3
result ← result % accumulator length
return result
```

arrival time, etc. to find distinct qualities with which to produce a recognizable uniqueness. What is missed in this approach is what lies below the surface, in the message body of the packet. We find that each packet produces traffic in which it identifies itself in various ways as it communicates with the device manufacturer, other devices, or other unknown IP addresses. During this activity devices often share their MAC address, current IP address, or other details that may specifically identify an individual device. Even if such identifying features are removed from the packet headers during processing to avoid bias in the machine learning algorithm (as they are in our study), information to a specific device may still appear in the encapsulated message body and can be captured by a hash regardless of encryption, etc. This is helpful when fingerprinting with LSH because the hash seeks to identify not specific information, but similarity to other packets produced by the same device. This type of communication is quite repetitive in IoT devices, and while subtle, can often appear as a consistent pattern in the feature vectors produced by FlexHash and are identifiable by the machine learning process. We believe this contributes heavily to the effectiveness of our approach.

D. Identification of IoT Devices

The overview of the IoT device identification system is presented in Figure 2. Traffic data is captured from IoT devices in a network and hashes of the traffic data are generated with FlexHash. These hashes are converted to feature vectors by converting each byte value in the resultant digest to base-10 numerical values ranging from 0-255. For instance, the byte value "FB" would be converted to the integer value 251. These feature vectors are then passed to our device identification system to train the underlying machine-learning model. Once a model is trained the identification system is ready for deployment on a router or micro-controller to serve as a gateway to a network. The device identification system will continuously monitor traffic data by capturing periodic samples

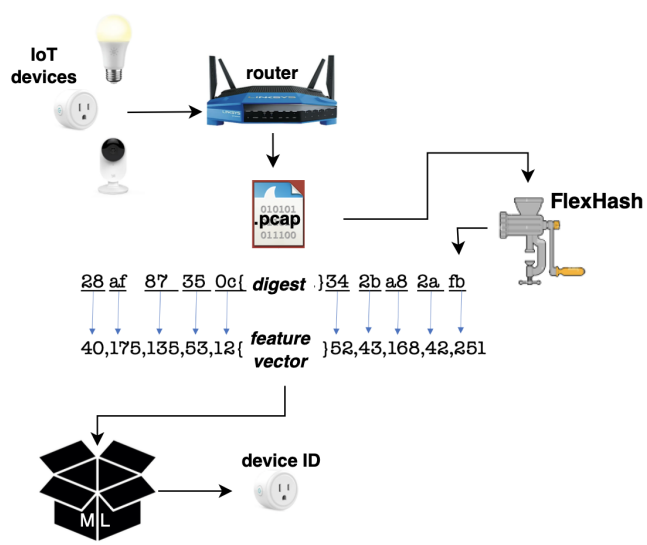


Fig. 2: The network traffic is continuously monitored by the device identification system. The traffic data is processed by FlexHash and converted into feature vectors. These feature vectors are given as input to the pre-trained ML model to identify the device that generated the traffic.

to identify known devices, new devices joining the network, and to identify changes indicating anomalous behavior. This is done by generating the digest of randomly captured traffic data in the form of packets, converting them to numeric feature vectors, and passing them to a pre-trained machine learning classifier.

E. Ensemble Learning

As a machine learning framework we use Autogluon-Tabular (AGT) [34] from Autogluon version 1.0.0. AGT is an automatic machine learning framework designed specifically for tabular datasets, such as spreadsheets. AGT aims to simplify the application of machine learning techniques for practitioners and researchers. We elect to use this framework because it enables streamlining the use of various best practices and machine learning strategies that lead to superior performance. The components of AGT that make it an effective framework for our data are the use of diverse base model types and carefully designed ensembling strategies.

AGT makes use of various well-known methods as base models. Base models are the underlying learning strategies used by AGT for modeling the input data. Each of these models has the capacity to perform well independently, but differences in their behavior lead to variance in the patterns recognized. Ensembling predictions from models with different perceptions of the data space leads to robust final predictions. Base models available are Random Forest, Extra Trees [35], CatBoost [36], LightGBM [37], XGBoost [38], Logistic Regression, K-Nearest Neighbors (KNN), and Neural Network. We select three base models to use in our experiments. These are Extra Trees, LightGBM, and XGBoost. These models were selected based on processing time and performance in preliminary experiments. Extra Trees, an ensemble learning method with additional randomness, builds multiple decision trees for predictions, potentially enhancing robustness at the cost of increased computational expense. LightGBM, a gradient boosting framework, prioritizes speed and efficiency with a tree-based learning approach, particularly suitable for large datasets but potentially more sensitive to overfitting. XGBoost, an optimized gradient boosting library, achieves exceptional performance through regularized learning and parallel computing, necessitating careful hyperparameter tuning.

Ensembling the predictions of multiple models leads to a reduction in the variance of a machine learning system and improved prediction accuracy [39]. AGT provides the implementation for two important forms of ensembling: repeated k-fold bagging and multi-layer stacking [40], [41]. Multi-layer stack ensembling is a strategy in which the output predictions of a preliminary layer of models are used as features or inputs to a subsequent layer. AGT's implementation also utilizes a form of skip connection which concatenates the original model features onto the preliminary layer's predictions. This technique allows the subsequent layers of models to have an understanding of the original data space in addition to the previous layers' predictions. Repeated k-fold bagging is a method that randomly splits the data into chunks and trains multiple models on different random chunks of the available data. When making a prediction, the input data is passed through each model and the outputs are voted on. Whichever output is most frequent is used as the output for the group. The final model produced by AGT is a combination of 68 total models in 2 layers. All bags contain 8 copies of the base model trained on different portions of the available data. The first layer (models trained on just input data) contains 2 Extra Trees models, 3 bags of LightGBM models with varying hyperparameters, and 1 bag of XGBoost models. The second layer is identical to the first, except models are trained with the output predictions of the first layer models concatenated to the original input data. AGT utilizes a method called ensemble selection [42], which takes predictions from all available models into account and produces a single output.

F. Dataset

For evaluation we focus on three categories of devices; smart plugs (*Ghome Smart Plug*), smart light bulbs (*General Electric CYNC Full-Color Smart Bulb*), and web cameras (*YI 1080p Home Camera*). Each category of device contains ***8 identical devices*** for a total of 24 devices. Devices are connected to the network and allowed to complete their initial setup phase and then traffic data is collected on a resting state for 24 hours. Data is then sanitized and header checksums are recalculated using *editcap*, replacing all MAC and IP addresses of the devices with 11:11:11:11:11:11 and 1.1.1.1 respectively to avoid bias introduced by unique addresses. Digests of each packet for each parameter combination are generated and converted to feature vectors as described in prior sections. For evaluation, the data was split into 80% for training and validation, and 20% for testing. Results are measured in terms of precision (i.e., $TP/(TP+FP)$), recall (i.e., $TP/(TP+FN)$), f1-score (i.e., $2/(1/precision + 1/recall)$), and accuracy (i.e., $(TP + TN)/(TP + FP + TN + FN)$) where TP, TN, FP, and FN stand for true positive, true negative, false positive and false negative.

IV. Experimental Analysis and Results

In this section, we analyze the performance of our device identification system using FlexHash with the following experiments.

- To demonstrate the system's ability to operate in a realistic network setting we consider the effect of adding background noise to each data set. To achieve this we combine device data with two types of noise: *IoT noise* (network traffic of random IoT devices), and *network noise* (random traffic from a live network).
- We generalize all devices into three categories: *smart plug*, *smart bulb*, or *web camera*. A model is built and packets from each device are classified by *genre*.
- We determine the source device of a network packet from a pool of 8 of the same model devices. We train a model, then identify packets from that group as belonging to a distinct individual.

TABLE III: Average performance in identifying device genre in the presence of noise and without noise.

Device	Accuracy		F1 Score	
	Without Noise	With Noise	Without Noise	With Noise
Smart Plugs	99.98	99.89	99.97	99.90
Smart Lights	99.88	99.41	99.92	99.57
Smart Cameras	99.99	99.98	99.99	99.98

- We compare individual device results to those from another popular LSH method, namely *Nilsimsa*.
- Finally, to show the effect of parameter tuning we generate confusion matrices and UMAP [43] visualizations of the data with the best and worst parameter sets.

A. Identify devices in the presence of background noise

In a live network traffic noise is inevitable. Thus, it is crucial to explore system performance in a noisy environment. This demonstrates the ability to apply device identification techniques in a realistic network setting when monitoring for IoT device membership or when searching for unknown devices by category. To achieve realistic results, we introduce two types of background noise: *IoT noise* (random IoT traffic from [4]) and *network noise* (random network traffic from a large set of heterogeneous devices. We create hashes of the noise data and add it to device data set labeled as either *network-noise* or *iot-noise*. Results show noise has a minimal impact on performance demonstrating the system's ability to function in a live network environment. Results for experiments with and without noise are presented in tables III, IV, and V.

B. Identify devices by genre

The identification of devices by genre enables inferring the device type for traffic captured in real-time via a single packet. By monitoring a network and testing random packets we can predict the likelihood that a particular packet belongs to a genre, i.e. the packet is *probably* a camera, or *probably* a smart bulb, etc. To do this, all 24 devices are re-labeled as either smart bulb, smart plug, or web camera, and a multi-class classifier is built. The implication here is that in a network scenario, randomly captured packets identified as being generated from some IoT device can be further categorized as a specific device type. This is useful for investigating unknown or rogue devices on a network that could potentially compromise security. Results for this experiment are presented in Table III. Results for identification by genre are above 99% for all devices both with and without background noise. We are also able to differentiate the noise type from known device genres, achieving an accuracy for *iot-noise* above 98% and for *network-noise* above 97%.

C. Identification of individual devices from identical peers

In this subsection, we work to identify an individual device from a group of identical peer devices. Instead of attempting to identify a heterogeneous set of unique devices, we address the more challenging task of identifying clusters of identical devices, a task under-explored in the literature. This type of identification is critical in tracking device behavior and membership as there are often multiple individuals of the same device appearing in a network. Experiments are performed both with and without background noise and results compared.

TABLE IV: Average performance in identifying identical devices from the same category in the presence of noise and without noise.

Device	Accuracy		F1 Score	
	Without Noise	With Noise	Without Noise	With Noise
Smart Plugs	85.79	85.02	85.77	84.91
Smart Bulbs	93.63	89.09	93.60	84.75
Web Cameras	97.89	98.61	97.78	98.55

Average results in terms of both accuracy and F1 score for this experiment are shown in Table IV. We see that without background noise, web cameras achieve an accuracy above 97% on average while smart bulbs and smart plugs achieve results above 85% and 93% respectively. We note that in the case of smart plugs, some imbalance was found in the data samples for a 24 hour period, with three of the eight plugs generating considerably more traffic than the other five, possibly accounting for this change in performance. Interestingly, devices with greater complexity appear to perform better than simpler devices when identifying an individual from identical peers. We note here that FlexHash and our device identification system achieve accuracy results in this more difficult scenario

TABLE V: Comparison of average performance, FlexHash vs Nilsimsa in identifying identical devices from the same category.

Device	Accuracy		F1 Score	
	FlexHash	Nilsimsa	FlexHash	Nilsimsa
Smart Plugs	**85.79**	72.97	**85.77**	73.27
Smart Bulbs	**93.63**	78.04	**93.60**	80.48
Web Cameras	**97.89**	84.74	**97.78**	84.66

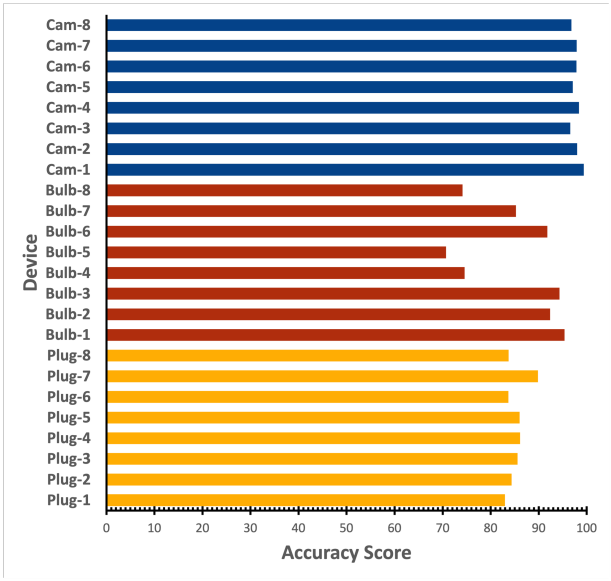
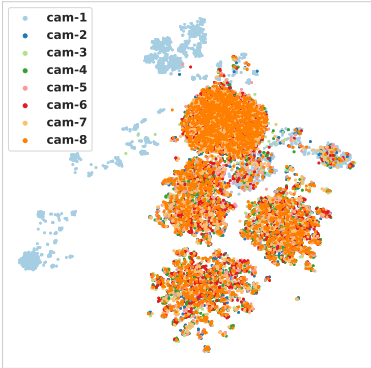


Fig. 3: Results for identification of individual devices from a group of identical peers (plugs, lights, and cameras).



(a) UMAP representation of camera data.

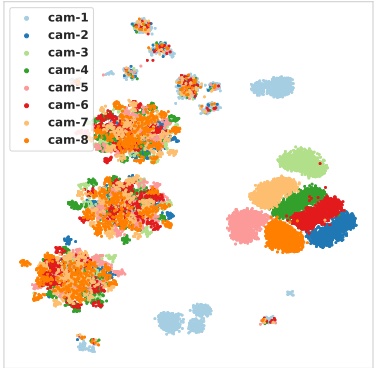
True labels \ Predicted Labels	cam-1	cam-2	cam-3	cam-4	cam-5	cam-6	cam-7	cam-8
cam-1	99.7%	0.0%	0.0%	0.1%	0.0%	0.1%	0.1%	0.0%
cam-2	2.7%	30.6%	17.4%	9.4%	9.7%	11.1%	10.9%	8.3%
cam-3	2.7%	6.0%	50.6%	8.5%	8.8%	9.4%	7.0%	7.0%
cam-4	2.6%	9.2%	18.4%	30.4%	8.7%	13.7%	9.4%	7.7%
cam-5	2.7%	7.9%	17.2%	7.7%	36.8%	9.7%	8.8%	9.2%
cam-6	2.8%	6.5%	17.8%	7.6%	10.5%	36.7%	10.9%	7.2%
cam-7	2.7%	8.5%	16.9%	7.9%	8.5%	12.1%	34.8%	8.6%
cam-8	2.8%	7.4%	18.1%	7.3%	12.6%	11.4%	10.7%	29.6%

(b) Confusion matrix for the task of identical device identification on cameras.

Fig. 4: UMAP representation of camera data (a) and confusion matrix for the task of identical device identification on cameras (b) hashed with accumulator length of 126, sliding window size of 6, and combination size of 6.

that are either competitive or superior to results offered by other approaches in the literature while using only a single packet sample. Results per device are shown in Figure 3. When running the same experiment with background noise, web cameras, smart bulbs, and smart plugs achieve an average accuracy of 98%, 89%, and 85% respectively. With noise added, we only see a slight degradation of performance in the smart bulbs, with web cameras and smart plugs performing at nearly an equal accuracy.

In Table V we compare the performance of FlexHash with another n-gram based LSH method, *Nilsimsa*. In our previous study, we observed that Nilsimsa outperforms other similar methods such as ssdeep [31], sdhash [32], and tlsh [33]. Thus, we can assume that better results with FlexHash over Nilsimsa will imply better results over other hashing techniques as well. Experiments are performed on network traffic data without background noise. FlexHash achieves an average accuracy of 97.74%, 93.63%, and 85.79% for web cameras, smart bulbs, and smart plugs respectively. On the same data, Nilsimsa achieves a lower performance of 84.74%, 78.04%, and 72.97%. This represents a percent increase of 13.00%, 15.59%, and 12.82%. We see here that in every case FlexHash achieves a significant increase in accuracy over Nilsimsa hashing, an indication of the effectiveness of FlexHash's tunable parameters.



(a) UMAP representation of camera data.

True labels \ Predicted Labels	cam-1	cam-2	cam-3	cam-4	cam-5	cam-6	cam-7	cam-8
cam-1	99.4%	0.2%	0.1%	0.1%	0.0%	0.1%	0.1%	0.1%
cam-2	1.3%	98.0%	0.1%	0.2%	0.1%	0.1%	0.1%	0.1%
cam-3	2.2%	0.2%	96.6%	0.2%	0.1%	0.1%	0.2%	0.6%
cam-4	1.2%	0.1%	0.1%	98.4%	0.1%	0.1%	0.1%	0.0%
cam-5	2.3%	0.1%	0.2%	0.1%	97.1%	0.1%	0.1%	0.1%
cam-6	1.7%	0.1%	0.1%	0.1%	0.1%	97.8%	0.1%	0.1%
cam-7	1.6%	0.1%	0.1%	0.1%	0.0%	0.1%	97.9%	0.1%
cam-8	2.0%	0.0%	0.9%	0.1%	0.0%	0.1%	0.1%	96.8%

(b) Confusion matrix for the task of identical device identification on cameras.

Fig. 5: UMAP representation of camera data (a) and confusion matrix for the task of identical device identification on cameras (b) hashed with accumulator length of 1024, sliding window size of 4, and combination size of 2

D. Visualization of FlexHash tunable parameters

In this section we demonstrate the strength of the FlexHash algorithm; hashing parameter tuning. As discussed above, our novel algorithm allows for the adjustment of variables that are pertinent to the hashing process (i.e. accumulator size, sliding window size, and combination size). The adjustment of these values allows FlexHash to capture more subtle aspects contained in the device network activity, producing hashes which are more effectively identified by the machine learning process when creating device traffic fingerprints.

To aid the reader in understanding differences in parameter settings we utilize two types of visual aids; confusion matrices and UMAP representations [43]. UMAP is a nonlinear dimensionality reduction technique used for visualizing high-dimensional data in lower-dimensional space while preserving local and global structure. It focuses on finding a low-dimensional representation of data points while maintaining their high-dimensional relationships with one another.

To demonstrate the effect of parameter tuning, we analyze the samples produced with the best and worst hashing parameters. Visualizations of these samples are found in Figure 4, 5. Note that the UMAP plots are generated with only the hashed data and have no exposure to the machine learning model. We begin by inspecting Figure 4 which reflects hashes generated from sub-optimal parameters. While cam-1 achieves high per-

formance, the clusters of data points representing cameras 2-8 are overlapping, thus the machine learning models struggle to accurately predict to which device a sample belongs.

In Figure 5(a) we see that by optimizing the parameter set used, FlexHash is able to hash devices into a representation which reliably separates data points originating from different devices into distinct spatial regions. The downstream effect of these parameters is shown in Figure 5(b), with all devices being classified correctly 96% or more of the time. Overall, locality-sensitive hashing removes the need for feature engineering in device fingerprinting, while tunable hash parameters enable adaptation of the hashing algorithm to various sets of IoT devices.

In the previous research on IoT device identification by network fingerprinting there are two key areas where there has been little work; sample data that includes the presence of background noise, and the identification of homogeneous (identical) devices. For a system to be useful in a live environment the inclusion of random and IoT-specific noise is essential as these will always be present. In the same way, network administrators will typically be required to manage groups of devices that are of the same manufacturer and even the same model, as in multiple sensors or cameras in a system. Similar to the works cited in our background study, our previous research [5] was effective in identifying heterogeneous devices, but was of limited use when facing homogeneous traffic. Through the tun-able parameters found in FlexHash characteristics in traffic produced by identical devices can be isolated and amplified by adjusting parameters such as accumulator length, ngram length, and combinations when hashing traffic. We believe this addresses a significant gap in the previous research.

V. Conclusion

With the increased presence of IoT devices in both private and commercial networks, new methods must be developed to track device membership and behavior to prevent the compromise of both devices and network data. Rogue or infected devices can lead to compromise by unauthorized users, and can be weaponized to launch attacks such as malware installation or distributed denial of service. As common identifiers such as MAC addresses are easily spoofed, methods such as network traffic fingerprinting are useful in providing management and security. While various methods have been proposed, many require the collection of specific traffic samples depending on size, device state, etc. to create rule sets, and samples must be compared to databases to match pre-defined fingerprints for identification. Others utilize machine learning approaches but require feature extraction and engineering, a process which can introduce considerable overhead, including the need for extensive domain knowledge in both machine learning and networking to develop feature vectors suitable for the chosen algorithms. In this paper, we introduce FlexHash, a hybrid locality-sensitive hashing approach that can convert unprocessed .pcap files into feature vectors without the need for feature extraction. FlexHash achieves a high degree of accuracy in network traffic fingerprinting and can successfully identify both heterogeneous and homogeneous devices as well as device genres in real-time by identifying single packets. In addition, background noise is included to demonstrate this system's ability to function in a live network setting to assist with network monitoring and anomaly detection.

References

[1] I. Ullah and Q. Mahmoud, "Network traffic flow based machine learning technique for iot device identification," in *2021 IEEE SysCon*. IEEE, 2021, pp. 1–8.

[2] R. Kolcun, D. Popescu, V. Safronov, P. Yadav, A. Mandalari, Y. Xie, R. Mortier, and H. Haddadi, "The case for retraining of ml models for iot device identification at the edge," *arXiv preprint:2011.08605*, 2020.

[3] R. Kolcun, D. Popescu, V. Safronov, P. Yadav, A. Mandalari, R. Mortier, and H. Haddadi, "Revisiting iot device identification," *arXiv preprint arXiv:2107.07818*, 2021.

[4] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A. Sadeghi, and S. Tarkoma, "Iot sentinel: Automated device-type identification for security enforcement in iot," in *2017 IEEE ICDCS*, 2017, p. 2177:2184.

[5] J. Thom, N. Thom, S. Sengupta, and E. Hand, "Smart recon: Network traffic fingerprinting for iot device identification," in *2022 IEEE CCWC*. IEEE, 2022, pp. 0072–0079.

[6] N.Naik, P.Jenkins, N.Savage, L.Yang, T.Boongoen, and N.Iam-On, "Fuzzy-import hashing: A static analysis technique for malware detection," *Forensic Sci. Int.: Digital Investigation*, vol. 37, p. 301139, 2021.

[7] V. Satuluri and S. Parthasarathy, "Bayesian locality sensitive hashing for fast similarity search," *arXiv preprint arXiv:1110.1328*, 2011.

[8] H. Noguchi, T. Demizu, N. Hoshikawa, M. Kataoka, and Y. Yamato, "Autonomous device identification architecture for internet of things," in *2018 IEEE WF-IoT*. IEEE, 2018, pp. 407–411.

[9] M. Mazhar and Z. Shafiq, "Characterizing smart home iot traffic in the wild," in *2020 IEEE/ACM IoTDI*. IEEE, 2020, pp. 203–215.

[10] M. Gill, D. Lindskog, and P. Zavarsky, "Profiling network traffic behavior for the purpose of anomaly-based intrusion detection," in *2018 IEEE TrustCom/BigDataSE*. IEEE, 2018, pp. 885–890.

[11] X. Feng, Q. Li, H. Wang, and L. Sun, "Acquisitional rule-based engine for discovering internet-of-things devices," in *2018 {USENIX}*, 2018, pp. 327–341.

[12] B. Charyyev and M. H. Gunes, "Iot traffic flow identification using locality sensitive hashes," in *IEEE ICC*, 2020, pp. 1–6.

[13] S. Aneja, N. Aneja, and M. Islam, "Iot device fingerprint using deep learning," in *2018 IEEE IOTAIS*. IEEE, 2018, pp. 174–179.

[14] Y. Meidan, M. Bohadana, A. Shabtai, J. Guarnizo, M. Ochoa, N. Tippenhauer, and Y. Elovici, "Profiliot: A machine learning approach for iot device identification based on network traffic analysis," in *Proceedings of the symposium on applied computing*, 2017, pp. 506–509.

[15] B. Bezawada, M. Bachani, J. Peterson, H. Shirazi, I. Ray, and I. Ray, "Iotsense: Behavioral fingerprinting of iot devices," *arXiv preprint arXiv:1804.03852*, 2018.

[16] S. Marchal, M. Miettinen, T. Nguyenand, A. Sadeghi, Ahmad-Reza, and N. Asokan, "Audi: Toward autonomous iot device-type identification using periodic communication," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1402–1412, 2019.

[17] O. Salman, I. H. Elhajj, A. Chehab, and A. Kayssi, "A machine learning based framework for iot device identification and abnormal traffic detection," *Transactions on Emerging Telecommunications Technologies*, p. e3743, 2019.

[18] A. Sivanathan, H. Gharakheili, and V. Sivaraman, "Inferring iot device types from network behavior using unsupervised clustering," in *2019 IEEE LCN*. IEEE, 2019, pp. 230–233.

[19] N. Ammar, L. Noirie, and S. Tixeuil, "Network-protocol-based iot device identification," in *2019 IEEE FMEC*. IEEE, 2019, pp. 204–209.

[20] ——, "Autonomous identification of iot device types based on a supervised classification," in *2020 IEEE ICC*. IEEE, 2020, pp. 1–6.

[21] S. Hamad, W. Zhang, Q. Sheng, and S. Nepal, "Iot device identification via network-flow based fingerprinting and learning," in *2019 18th IEEE TrustCom/BigDataSE*. IEEE, 2019, pp. 103–111.

[22] A. Aksoy and M. Gunes, "Automated iot device identification using network traffic," in *2019 IEEE ICC*, 2019, pp. 1–7.

[23] L. Fan, S. Zhang, Y. Wu, Z. Wang, C. Duan, J. Li, and J. Yang, "An iot device identification method based on semi-supervised learning," in *2020 IEEE CNSM*. IEEE, 2020, pp. 1–7.

[24] B. Desai, D. Divakaran, I. Nevat, G. Peter, and M. Gurusamy, "A feature-ranking framework for iot device classification," in *2019 IEEE COMSNETS*. IEEE, 2019, pp. 64–71.

[25] V. Roussev, "Hashing and data fingerprinting in digital forensics," *IEEE Security & Privacy*, vol. 7, no. 2, pp. 49–55, 2009.

[26] W. P. Filho, C. Ribeiro, and T. Zefferer, "An ontology-based interoperability solution for electronic-identity systems," in *2016 IEEE SCC*. IEEE, 2016, pp. 17–24.

[27] M. Pirker, P. Kochberger, and S. Schwandter, "Behavioural comparison of systems for anomaly detection," in *Proceedings of the 13th International Conference on Availability, Reliability and Security*, 2018, pp. 1–10.

[28] H. Kim, H. Song, J. Seo, and H. Kim, "Andro-simnet: Android malware family classification using social network analysis," in *2018 IEEE PST*. IEEE, 2018, pp. 1–8.

[29] E.Damiani, S.Vimercati, S.Paraboschi, and P.Samarati, "An open digest-based technique for spam detection." *ISCA PDCS*, vol. 2004, p. 559:564, 2004.

[30] M. Marsono, "Packet-level open-digest fingerprinting for spam detection on middleboxes," *International Journal of Network Management*, vol. 22, no. 1, pp. 12–26, 2012.

[31] N.Sarantinos, C.Benzaid, O.Arabiat, and A.Al-Nemrat, "Forensic malware analysis: The value of fuzzy hashing algorithms in identifying similarities," in *Trust/BigDataSE/ISPA*. IEEE, 2016, pp. 1782–1787.

[32] F. Breitinger and H. Baier, "Properties of a similarity preserving hash function and their realization in sdhash," in *2012 Information Security for South Africa*. IEEE, 2012, pp. 1–8.

[33] J. Oliver, C. Cheng, and Y. Chen, "Tlsh–a locality sensitive hash," in *2013 IEEE CTC*. IEEE, 2013, pp. 7–13.

[34] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola, "Autogluon-tabular: Robust and accurate automl for structured data," *arXiv preprint arXiv:2003.06505*, 2020.

[35] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, p. 3–42, 2006.

[36] A. V. Dorogush, V. Ershov, and A. Gulin, "Catboost: gradient boosting with categorical features support," *CoRR*, vol. abs/1810.11363, 2018. [Online]. Available: http://arxiv.org/abs/1810.11363

[37] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 3149–3157.

[38] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[39] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 1–15.

[40] P. W. M. BAMBANG PARMANTO and H. R. DOYLE, "Reducing variance of committee prediction with resampling techniques," *Connection Science*, vol. 8, no. 3-4, pp. 405–426, 1996. [Online]. Available: https://doi.org/10.1080/095400996116848

[41] K. M. Ting and I. H. Witten, "Stacking bagged and dagged models," in *Proceedings of the Fourteenth International Conference on Machine Learning*, ser. ICML '97. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997, p. 367–375.

[42] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, "Ensemble selection from libraries of models," in *Proceedings of the Twenty-First International Conference on Machine Learning*, ser. ICML '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 18. [Online]. Available: https://doi.org/10.1145/1015330.1015432

[43] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," 2020.